

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I G.Karthik Reddy-192525047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Karthik Reddy

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

EVALUATION RUBRICS:

Criterion	Weighta ge (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.

4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.

8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

1. Medical Diagnosis Expert System

Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and clinical information using production rules, forward and backward chaining.

1. Problem Analysis

Problem Statement

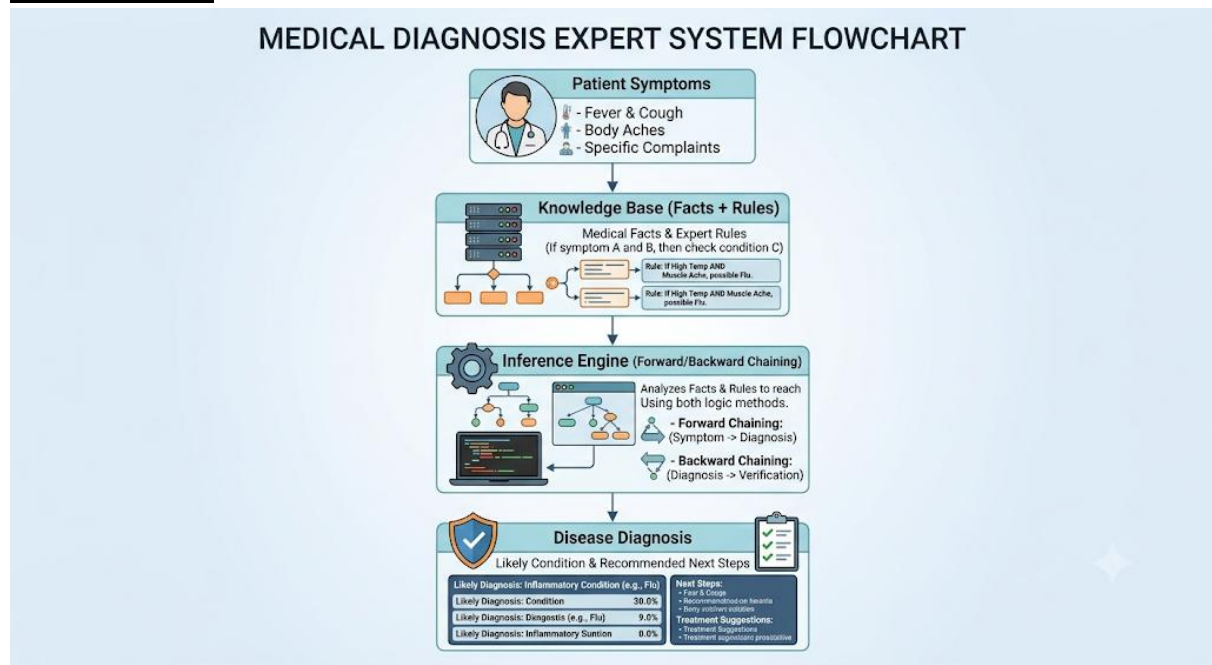
Hospitals and clinics receive patients with various symptoms. The system should analyze:

- Fever
- Cough
- Headache
- Body Pain
- Sore Throat
- Fatigue

and identify diseases such as:

- Common Cold
- Flu
- COVID-19
- Malaria

Domain Model



2. Knowledge Acquisition

The knowledge is collected from:

- Medical books
- Doctor consultation
- WHO guidelines
- Clinical symptom patterns

3. Knowledge Representation

Knowledge is represented as:

Facts

symptom(john, fever).
symptom(john, cough).
symptom(john, headache).
symptom(john, body_pain).

symptom(ravi, fever).
symptom(ravi, chills).
symptom(ravi, sweating).

4. Production Rules

disease(Patient, flu) :-
 symptom(Patient, fever),
 symptom(Patient, cough),
 symptom(Patient, body_pain).

disease(Patient, cold) :-
 symptom(Patient, cough),
 symptom(Patient, sore_throat).

disease(Patient, covid19) :-
 symptom(Patient, fever),
 symptom(Patient, cough),
 symptom(Patient, fatigue).

disease(Patient, malaria) :-
 symptom(Patient, fever),
 symptom(Patient, chills),
 symptom(Patient, sweating).

5. Complete SWI-Prolog Program

/* Facts */

symptom(john, fever).
symptom(john, cough).
symptom(john, headache).
symptom(john, body_pain).

symptom(ravi, fever).
symptom(ravi, chills).
symptom(ravi, sweating).


```
symptom(anu, fever).  
symptom(anu, cough).  
symptom(anu, fatigue).
```

```
/* Rules */
```

```
disease(Patient, flu) :-  
    symptom(Patient, fever),  
    symptom(Patient, cough),  
    symptom(Patient, body_pain).
```

```
disease(Patient, cold) :-  
    symptom(Patient, cough),  
    symptom(Patient, sore_throat).
```

```
disease(Patient, covid19) :-  
    symptom(Patient, fever),  
    symptom(Patient, cough),  
    symptom(Patient, fatigue).
```

```
disease(Patient, malaria) :-  
    symptom(Patient, fever),  
    symptom(Patient, chills),  
    symptom(Patient, sweating).
```

```
/* Recommendation */
```

```
advice(Patient, rest) :-  
    disease(Patient, flu).
```

```
advice(Patient, doctor_visit) :-  
    disease(Patient, covid19).
```

```
advice(Patient, blood_test) :-  
    disease(Patient, malaria).
```

6. Forward Chaining

Forward chaining starts from facts and derives conclusions. The document specifically requires demonstration of forward chaining.

Facts

John:

Fever

Cough

Body Pain

Rule Matching

IF Fever

AND Cough

AND Body Pain

THEN Flu

Inference

John $\rightarrow$ Flu

John $\rightarrow$ Rest

Forward Chaining Diagram

Facts

↓

Rules

↓

Disease

↓

Advice

7. Backward Chaining

Backward chaining starts from the goal and checks facts. The assessment requires goal-driven backward chaining using Prolog.

Query

?- disease(john, flu).

Prolog Checks

Is fever present? YES

Is cough present? YES

Is body_pain present? YES

Result

true.

Another Query

?- disease(ravi, malaria).

Output

true.

8. Unification

Unification is the process of matching variables with facts.

Example:

disease(Patient, flu)

If:

Patient = john

then:

disease(john, flu)

becomes true.

9. Backtracking

Example query:

?- disease(john, X).

Prolog searches:

X = flu

If another rule matches, Prolog automatically checks it through backtracking.

Example:

X = flu

X = covid19

(if all symptoms satisfy both rules).

10. Procedural vs Non-Procedural Paradigm

The project requires comparison of procedural and non-procedural paradigms.

Feature	Procedural Language	Prolog (Non-Procedural)
Focus	Steps to solve	What is true
Programming Style	Algorithm based	Logic based
Example	C, Java	Prolog
Control	Programmer	Prolog engine
Reasoning	Manual	Automatic

Example

Procedural:

Read symptoms

Check conditions

Find disease

Display output

Prolog:

disease(Patient, flu) :-

 symptom(Patient, fever),

 symptom(Patient, cough),

 symptom(Patient, body_pain).

11. Test Cases

Patient	Symptoms	Predicted Disease
John	Fever, Cough, Body Pain	Flu
Ravi	Fever, Chills, Sweating	Malaria
Anu	Fever, Cough, Fatigue	COVID-19

12. Sample SWI-Prolog Execution

?- disease(john, X).

X = flu.

?- disease(ravi, X).

X = malaria.

?- advice(john, X).

X = rest.

?- advice(ravi, X).

X = blood_test.

13. Advantages

- Fast diagnosis
- Automatic reasoning
- Easy knowledge representation
- Supports forward chaining
- Supports backward chaining
- Uses unification and backtracking
- Explainable AI system

14. Conclusion

The **Medical Diagnosis Expert System** developed using **SWI-Prolog** demonstrates:

- ✓ Facts and production rules
- ✓ Knowledge representation
- ✓ Forward chaining
- ✓ Backward chaining
- ✓ Unification
- ✓ Backtracking
- ✓ Declarative programming